© 2027 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

TwinHash: Two-Sided Object Identity Management for Time- and Energy-Efficient Agentic Workflows

Sangbu Yun[1], Jung Gyu Min[2], Jin Hyun[2], Jeonghwan Ahn[1], and Youngjoo Lee[1]
[1]KAIST, [2]POSTECH
{boo, jhahn09, youngjoo}@kaist.ac.kr, {minjg.epiclab, jinhyun.epiclab}@gmail.com

arXiv:2611.09254v1 [cs.AR] 6 Nov 2026

***Abstract*—Agentic artificial intelligence (AI) services interleave large language model (LLM) generation with external tool execution, and increasingly rely on search-based workflows that repeatedly revisit shared reasoning states and computations. These patterns create substantial redundancy across tool outputs, LLM responses, and key-value (KV) prefixes, yet existing serving systems primarily optimize shared prompt prefixes, while naive input-keyed caching is unsafe when identical requests produce different outputs. We present TwinHash, a unified object management architecture that assigns each object an input-side hash key to recognize repeated requests and an output-side content fingerprint to validate and locate stored state, complemented by type-specific reuse guards that preserve correctness and response diversity. A software object identity table maintains reusable objects across requests and trajectories, while a hardware object placement table tracks their residency across hierarchical memory tiers so that evicted state can be retrieved rather than recomputed. Across seven agent frameworks and five benchmark suites, reuse follows workflow structure more than benchmark identity. TwinHash serves up to 81% of tool outputs wherever deterministic tool reads recur, and up to 92% of LLM responses in search-based frameworks. Retrieving an evicted KV-prefix from DRAM reduces TTFT by up to 5.6× on Qwen3-32B, with greater gains projected for larger dense models. Under search-based frameworks, TwinHash reduces end-to-end wall-clock time by up to 87% and measured task energy by up to 81% over a prefix-cached baseline, while a per-key serving cap controls the quality-reuse trade-off in adaptive search. These results demonstrate that unified identity, safety, and placement management can convert structural redundancy in emerging agentic workflows into substantial serving-efficiency gains.**

I. Introduction

Nowadays, large language models (LLMs) have emerged as core building blocks for coding, question answering, and knowledge-intensive tasks [6], [42]. Techniques like chain-of-thought (CoT) and tree-structured prompting elevate single-step generation into multi-step reasoning traces [54], [60]. Furthermore, agentic workflows introduce tool use with an outer action loop where the model issues API calls, executes code, or queries a retriever, conditioning its subsequent steps on the resulting observations [48], [50], [61]. Multi-agent systems further scale these loops by distributing them across role-specialized collaborators [19], [30], [44], [55]. Through the progression of agentic workflows, the execution graph of a request is increasingly built at runtime rather than fixed in advance. Consequently, benchmarks such as AgentBench, SWE-bench, and τ-bench evaluate agents on long-horizon tasks requiring multiple rounds of tool feedback rather than isolated, single-turn prompts [23], [32], [56], [59].

The transition into tool-augmented service offloads computational burden from the model itself onto external tools. Recent agent frameworks consequently spend a large fraction of their end-to-end wall-clock time waiting on tool execution rather than on model inference [25], [50], [61], [65]. Modern frameworks exacerbate this overhead by adaptively widening inference-time tree searches [22] or searching over entire workflow programs [63], thereby increasing the algorithmic fan-out that multiplies both tool execution and LLM decoding costs. Consequently, dynamic and tool-augmented reasoning in agentic workflows substantially inflates energy and infrastructure demands compared to conventional LLM services [24].

The shift toward dynamic workflows presents a fundamental serving problem beyond mere application-level concerns. Conventional serving schedulers treat tool calls as opaque external waits, causing tool latency to block the critical path while GPU resources idle [5], [31], [36], [51]. Along with LLM inference, which dominates the footprint of deployed agent services [24], [25], [61], growing tool-layer workloads further complicate cost optimization, including latency and energy.

Modern agentic workflows rarely resolve tasks via single-shot trajectories. Instead, advanced frameworks rely on iterative trials, result verification, and alternative approach exploration [22], [25], [56], [59], [63], [65]. Meanwhile, agents frequently invoke identical tools and LLM decode requests using identical arguments. The duplicated executions emerge across retries, parallel search branches, and distinct tasks sharing identical state. Empirical analysis shows that 31-95% of tool outputs constitute byte-exact duplicates, and 48-94% of submitted LLM prompts match previous requests across various agentic frameworks [22], [59], [63], [65]. The observed redundancy manifests across three distinct computational artifacts, including tool invocations, LLM decode requests, and LLM prefill operations. Re-executing the redundant operations incurs severe systemic overhead, wasting CPU and GPU resources, network bandwidth, and external API budgets.

Standard caching mechanisms represent the most basic approach to exploit computational redundancy. However, conventional input-keyed caches fail in agentic workflows for two critical reasons. First, identical queries submitted to external tools frequently produce divergent outputs. For instance, database lookups yield strictly deterministic byte sequences, whereas 31-100% of repeated retrieval-augmented queries return different results. Second, massive KV tensor volumes rapidly force cache evictions from hot-tier HBM. The eviction

triggers the recomputation of massive duplicate objects occupying gigabyte-scale storage. Conventional systems blindly regenerate evicted data even when identical bytes remain untracked within lower-tier memory subsystems.

Our core insight is an architecture that assigns every object a two-sided hash identity. The *input-side* key functions as a deterministic execution signature, which exclusively drives the elimination of redundant computations. The key generation applies uniformly across *objects*, including tool outputs, model responses, and KV-segments. The *output-side* content fingerprint functions as a physical address key and data integrity validator, derived from an output byte sequence. The underlying system indexes physical addresses using the content fingerprint, preventing physical replication. Moreover, the fingerprint encourages memory pressure to demote objects to lower-tier memory instead of outright deletion. Conversely, the fingerprint verifies the integrity of objects promoted to upper-tier memory, so corrupt or misplaced bytes are not served. Further validity rules prevent stale reuse.

To this end, we present TwinHash, a unified hash-based object cache designed for dynamic agentic workflows. TwinHash registers every materialized tool and LLM output in an identity table keyed by an input-side hash, validates candidate hits via an output-side fingerprint, and tracks object residency across the memory hierarchy to fetch rather than recompute evicted data. Consequently, the system jointly recovers tool-worker CPU cycles, network bandwidth, API budgets, and GPU compute time. Workflow structure governs the savings, as recurring deterministic tool reads benefit any framework, whereas response reuse requires search-based execution.

This paper makes the following primary contributions:

- **A characterization of reusability in agentic workflows.** We show that potential reuse heavily depends on agent frameworks instead of their tasks (benchmarks). Tool output reuse follows tool-request recurrence and is bounded by tool determinism, whereas LLM response repetition is confined to search-based policies with hit rates of 14-92% while single-trajectory policies reach near zero.
- **A safe, content-derived identity mechanism.** We introduce a two-sided hash architecture TwinHash that captures structural redundancies including cross-task overlaps invisible to prefix- or session-scoped caches. The reuse guards prevent cache poisoning and thus misuse of cached objects. The validity model adds tool versions, per-resource state epochs, and sampled audits. In replays with database and file drift, the model serves no stale object when change signals precede dependent reads.
- **Unified resource savings and robustness under memory pressure.** We demonstrate substantial resource savings in tool outputs, LLM responses, and KV-prefixes using a unified caching abstraction. The proposed method successfully maintains the reuse hit rate under severe memory pressure thanks to its separated identity tables. In live paired deployments of search-based frameworks, TwinHash reduces end-to-end serving latency by up to 87% and measured task energy by up to 81%.

II. Background

A. LLM Serving and Agentic Workflows

LLM inference proceeds in two phases. The *prefill* phase processes input tokens in parallel and materializes their key and value projections in a KV cache, whereas the *decode* phase autoregressively generates output tokens using the populated cache [53]. To maximize throughput, serving systems rely on continuous batching and fine-grained KV-cache management, as in vLLM's PagedAttention [27]. For agentic workloads, this design depends on maintaining a sufficiently large pool of ready-to-run model requests. External tool invocations suspend individual trajectories and shrink this pool; when the remaining request concurrency is insufficient, batch occupancy decreases and GPU resources become underutilized [24].

Fig. 1 illustrates the architectural evolution that gives rise to this utilization challenge. Standard LLM deployments expose a simple request-in, text-out interface [6], [42] (❶), while reasoning techniques extend each request with a longer internal derivation that trades additional decode compute for improved accuracy [54], [60] (❷). Agentic frameworks then place this generation process inside an outer control loop that interleaves model reasoning with external actions [48], [50], [61]. *Single-trajectory* frameworks follow one evolving execution path or commit to a statically planned graph, as in ReAct's reason-act loop and LLMCompiler's plan-once execution [25], [61] (❸). *Search-based* frameworks instead explore multiple candidate continuations from shared intermediate states, spanning LATS's tree search, AB-MCTS's adaptive branching, and AFlow's workflow-level optimization [22], [63], [65] (❹). Role-specialized multi-agent organization [19], [30], [44], [55] is orthogonal to this execution-structure axis, as either family can distribute its graph across collaborating agents.

Transitioning from a naive single-LLM policy to a richer agentic workflow can substantially improve task quality, as demonstrated in Fig. 2. These quality gains, however, come with increased structural complexity and thus higher service costs. For instance, multi-step retrieval-and-verification workflows improve answer F1 scores on multi-hop QA benchmarks from 0.12 to 0.48 on HotpotQA, from 0.14 to 0.38 on MuSiQue, and from 0.19 to 0.49 on 2WikiMultiHopQA [18], [52], [57]. Dynamic workflows further determine portions of their execution graph mostly at runtime, introducing irregular model and tool invocations that are difficult to batch efficiently [24]. The two architectural families expose redundancy in structurally different forms. Within a single-trajectory framework (❸), successive model invocations typically append new observations to a largely shared context, producing substantial KV-prefix overlap even when their complete prompts differ. Such trajectories may also revisit deterministic tool state as reasoning and verification proceed. A search-based framework (❹) additionally branches from common intermediate states, causing multiple candidates to reuse long context prefixes and repeat identical or equivalent model and tool requests [4], [35], [65]. As inference-time scaling matures,

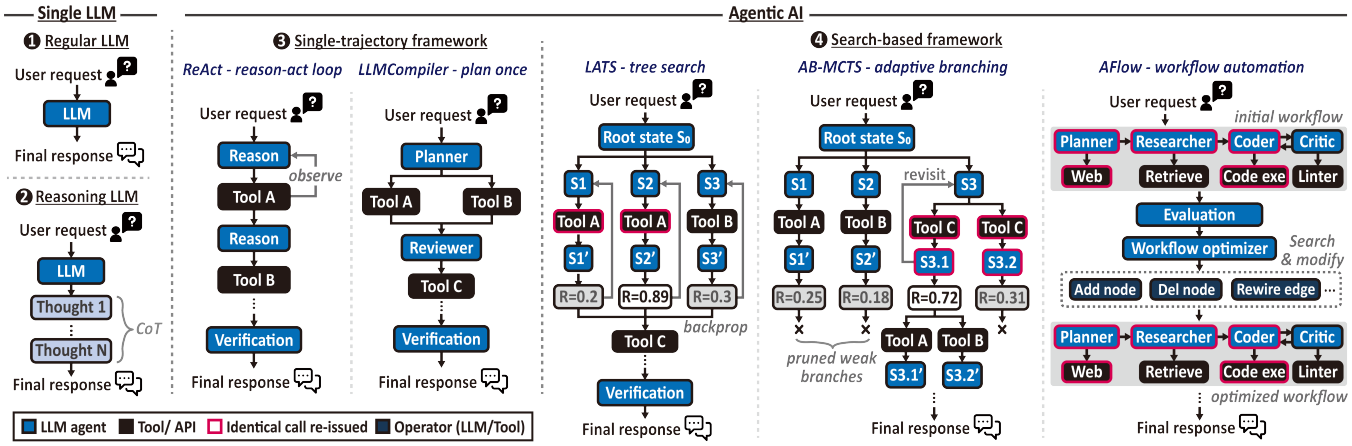

Fig. 1. Evolution from single-call LLM serving to agentic workflows. Reasoning extends each request into multi-step generation, while tool use introduces external actions; agentic frameworks organize these steps as either a single trajectory or a search over shared states.

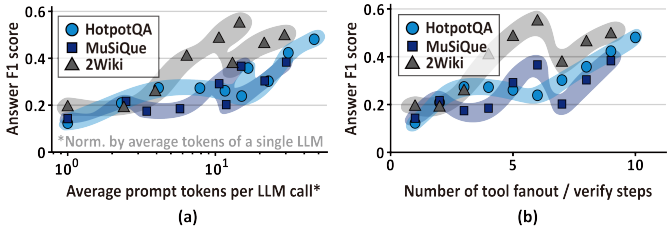

Fig. 2. Quality versus workflow complexity on multi-hop QA [18], [52], [57], measured with Qwen3-14B. Answer F1 increases with (a) the average prompt tokens per LLM call and (b) the number of tool fan-out and verification steps.

TABLE I
COMPARISON OF EXISTING REUSE SYSTEMS WITH HOW THEY IDENTIFY AND GUARD REUSABLE STATE.

System class	Reuses	Keyed by	Cross-req.	Exact
Prefix cache [27], [64]	KV prefix	position	prefix	exact
KV offload [33], [45]	KV blocks	prefix	prefix	exact
Workflow sched. [31], [43]	KV, schedule	step graph	program	exact
Semantic cache [3]	responses	similarity	any	none
Build cache, CAS [10], [11]	outputs	input / content	any	assumed
TwinHash	tool/resp./KV	input + content	any (tool)	verified

recent systems are increasingly adopting such search-based execution architectures [2], [15], [21], [22], [62], [63].

B. Existing Serving Optimizations

Recently, extensive research has focused on optimizing the LLM-side components of agentic workloads as in Table I. Prefix caches [27], [64] and KV-offload [33], [45] stacks reuse or relocate prefill state keyed by token position, and workflow-aware methods [31], [43] extend the same state by prefetching, overlapping, or scheduling it along predicted execution paths. Semantic caches instead key on embedding similarity to serve a broader range of requests [3]. Cache-augmented generation (CAG) [7] exploits predefined knowledge sources to persist state across queries. Prior work, however, is mostly limited to optimizing the prefill phase of LLM services and agentic workflows. GPTCache [3] reuses whole LLM responses for similar requests, yet it prioritizes similarity over exactness and cannot optimize internal flows of agentic services. Hence, tool invocations and LLM decode responses were not considered as sources of reusable content while preserving accuracy.

III. Motivation and Characterization

Targeting efficient agentic service, we motivate TwinHash through four key observations from various workflows. First, tool and decode layers represent a substantial yet unmanaged cost (ⓐ). Second, tool and decode outputs exhibit significant redundancy that current serving systems fail to reclaim (ⓑ). Third, the safety of output reuse depends on a determinism spectrum that naive caches cannot accommodate (ⓒ). Finally, a stable identity mechanism alone is insufficient once reusable objects are evicted from the hot memory set (ⓓ).

ⓐ Tool and decode layers represent a substantial, unmanaged cost. Fig. 3 shows the latency breakdown between tool, decode, and prefill time across agent frameworks and benchmarks with radix prefix caching [64]. Tool time spans from issuing a tool call until its result returns, and a verified hit replaces the interval with a lookup. As agent capabilities expand, agents spend an increasing fraction of their time invoking external tools and LLM decode rather than prefill. Extensive research on KV-prefix reuse exacerbates the imbalance as the prefill share shrinks further. On several tool-augmented tasks, tool and decode consume 2.1-61.7% and 37.0-99.9% of end-to-end wall-clock time, respectively. First, tool calls are serialized in conventional schedulers against model generation, which blocks GPU utilization during external execution [5], [24], [36], [51]. The tool work itself also consumes CPU cycles, network bandwidth, and external API budgets. Second, LLM decoding consumes a substantial fraction of overall latency due to its autoregressive nature, which drains GPU resources. These resource demands scale with the call count, and Fig. 4 shows the profiled agents issuing up to 152 tool and 151 LLM calls per task. Despite this footprint, current serving stacks fail to treat these outputs as managed state.

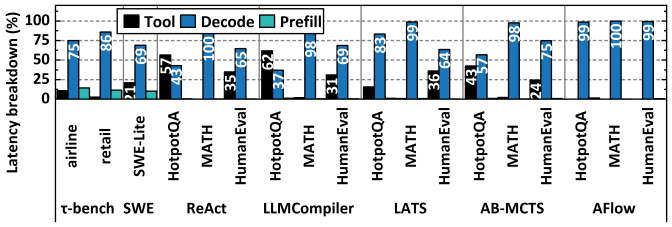

Fig. 3. Latency breakdown across tool, decode, and prefill for agent frameworks and benchmarks measured on Qwen3-14B with prefix caching [64].

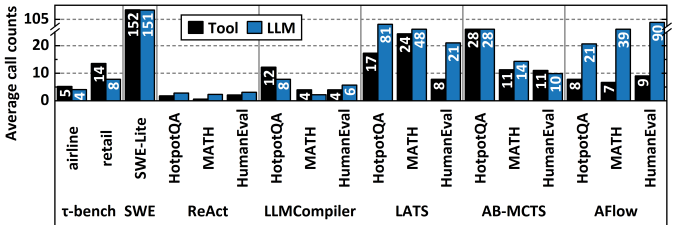

Fig. 4. Average number of tool and LLM calls per task measured on Qwen3-14B over 10-20 tasks per experimental cell.

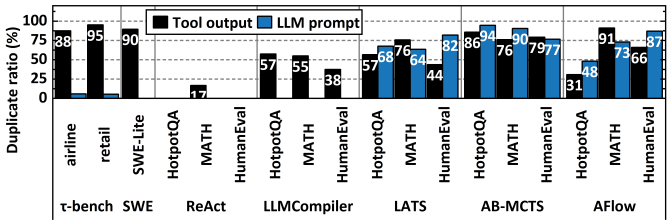

Fig. 5. Duplicate rates in tool outputs and LLM prompts. Single-trajectory frameworks (τ-bench, SWE, ReAct, and LLMCompiler) exhibit near-zero LLM-prompt duplication, whereas search-based frameworks (LATS, AB-MCTS, and AFlow) repeat both tool outputs and LLM prompts. Tool bars count outputs identical to any earlier one (76% for SWE without empty outputs). LLM bars count requests that repeat except for the seed.

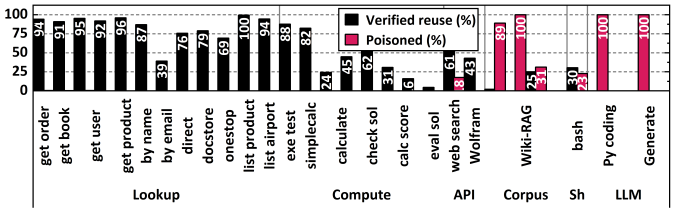

Fig. 6. Verified reuse and cache-poisoning rates across heterogeneous operators. Poisoned objects correspond to repeated arguments whose outputs differ and would therefore be served incorrectly by an argument-only cache.

❶ Tool and decode outputs exhibit massive redundancy. Redundancies arise when agents re-issue identical queries, parallel search branches access the same state, or distinct tasks query shared records. In Fig. 5, up to 95.1% and 94.0% of tool outputs and LLM prompts were duplicates, respectively. Tool output repetition appears in every agent framework, whereas LLM prompt repetition emerges mostly under search-based execution. The main reason for the repetition is structural split and branch expansion that re-issues identical contexts for both tool and LLM. Re-execution costs much more than serving known objects: a database (DB) query consumes 9.1× more energy than fetching a cached result, and recomputing KV uses 70× more GPU energy than fetching from DRAM.

❷ The safety of reuse follows a determinism spectrum. Fig. 6 shows the verified tool and LLM output reuse rate and the error rate of an argument-only cache (*poisoned*), which is keyed on the call arguments only without checking the results. Deterministic DB lookup tools are verified as reusable for 87.9% of their repeats on average with zero poisoning. Conversely, retrieval-augmented queries [29] and inherently non-deterministic calls including web search, LLM coding and LLM generation suffer poisoning rates of 18-100%. Moreover, executing the same shell commands across different repositories consistently fails as their environment has changed (shell exec). Therefore, object reuse within an agentic trajectory requires a solid method to prevent reuse beforehand and verify output content to avoid silent corruption from misuse.

❸ Identity mechanisms require placement metadata under memory pressure. Identifying a reusable object yields no benefit if its bytes have already been evicted. Severe memory pressure typically arises during long multi-turn sessions or CAG usage, where huge KV-cache objects quickly exhaust the hot memory tier. On various benchmarks using Qwen3-14B, Fig. 7 shows that decreasing resident capacity degrades the reusable-object hit rate and forces recomputation. This pressure intensifies under frontier-scale models such as Llama 3.1 (405B) and Kimi K3 (2.8T-A104B) [14], [26], whose weights and KV caches crowd the same memory hierarchy. Augmenting the object identity with hierarchical placement metadata allows the runtime to fetch evicted data from lower tiers. Hence, a robust cache should manage content identity with its residency to remain effective beyond the hot set.

These observations set three requirements for TwinHash.

1) Unified caching mechanism for heterogeneous and non-deterministic operators. The proposed design must consider two independent features simultaneously: a unified input-side caching key for heterogeneous objects from either tool or LLM, and reuse safety determined by invocation nature.

2) Management for physical address with its identity. An object's residency and physical location metadata must remain coupled with its identity, so that the runtime fetches known objects from lower memory tiers rather than recomputing them once the hot tier reaches capacity.

3) Reuse policy depends on frameworks rather than tasks. The agent framework dictates reuse potential far more than the dataset, as re-issues mostly expose tool and response reuse. Re-issues that seek diversity from non-determinism should be protected to preserve the framework's service quality.

IV. TwinHash Architecture Design

TwinHash introduces a unified object-management layer at the execution boundaries where an agent's workflow generates reusable objects. These boundaries include the external tool interface, the model response for structurally repeated prompts, and the KV-segment interface. Every eligible request follows

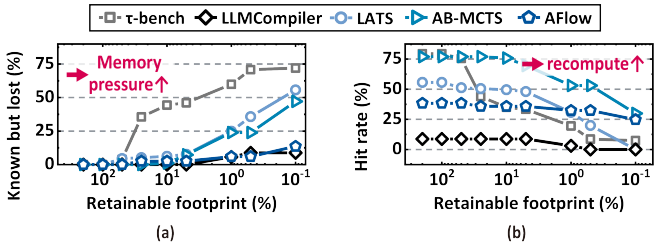

Fig. 7. Effect of memory pressure on reusable objects. (a) Known-but-lost fraction and (b) reusable object hit rate as the retainable footprint shrinks.

the same path regardless of type: the runtime canonicalizes the request into a descriptor, digests it into a lookup key, probes a central object directory, and either serves a validated hit or executes natively and commits the fingerprinted output. A descriptor is the set of structured fields that determine an object's content, listed for each object type in §IV-A.

Fig. 8 illustrates the TwinHash unified execution path, which separates object management into two caching data structures. The software-level object identity table (OIT) tracks the existence of requested objects using hash-generated keys and content fingerprints, while the hardware-level object placement table (OPT) records the physical location of each object across the memory hierarchy. Since both OIT and OPT are separated from the object bytes, the system identifies objects' existence and residency even after demotion to a lower memory tier.

A. Two-Sided Hash Identity and OIT

The input-side identity mechanism constructs lookup keys from semantic meaning rather than framework-specific syntax, capturing cross-agent redundancies across heterogeneous reusable objects. The descriptor fields adapt to each underlying object type (❶). For a *tool invocation*, the descriptor comprises task scope, tool name and version, regularized tool input, and the state epochs of the resources that the call reads, as detailed in §IV-B. The regularization removes transient runtime fields so that semantically identical calls produce identical keys. For an *LLM response*, the descriptor incorporates model identifier, prompt messages, and sampling parameters. Note that the random seed and the request identifier are excluded to prevent fresh seeds from obscuring the structural repetitions of search-based policies. Sampled diversity is instead managed during the serving phase, as described in §IV-D. For a *KV-segment*, the descriptor captures model configuration, exact token IDs, and absolute position range. Hence, the KV key remains position-bounded unlike tool and response keys.

The runtime hashes each descriptor using 128-bit BLAKE2b and probes a native matching directory (❷). The proposed OIT is designed as a growable hash table in host DRAM and thus has no set conflicts (❸). Per-task identities expire at task end, and shared identities are bounded at 3 MB (64K entries) under LRU, as in SGLang, vLLM, and LMCache [27], [33], [64]. Hence, an eviction only forgoes a later reuse. Fig. 9 shows the total number of OIT entries under various workload evaluations, which require at most 1.4K entries.

The output-side identity mechanism manages reuse of the stored object by fingerprint. When a tool or model is invoked for the first time, the OIT lookup misses, so TwinHash fingerprints the output with 128-bit BLAKE2b (❹) and commits the key and fingerprint to the OIT. On a hit, TwinHash retrieves the object's physical address and metadata from the OPT (❺). The fingerprint verifies the retrieved object is valid, as external programs might have mutated the address. Keys producing an identical fingerprint also collapse into a single copy.

B. Reuse Safety and Reuse-Guard

As analyzed in §III, the determinism spectrum for heterogeneous operators, including tools and LLMs, must be considered when classifying the redundancy. Hence, we introduce an independent structural scoping method *reuse-guard* to manage reuse safety throughout the agentic service (❻).

❶ Tool reuse safety. Tool reuse becomes unsafe for two distinct reasons: a tool might be inherently non-deterministic, and even an output from a deterministic tool might become outdated or invalid as state changes. First, *SafeTools* is constructed through fingerprint-based determinism profiling before active reuse. The profiling pass initially admits candidate read-only tools but executes every invocation natively. If an identical input-side key produces nonunique fingerprints, the tool is classified as nondeterministic and excluded from SafeTools. Profiling runs once per tool version, which is part of the key. Between versions, sampled audits re-execute 5% of the hits of profiled tools off the critical path, and a single mismatch excludes the tool until its next version. The audits thus detect a wrong declaration within 20 wrong hits on average, without added latency and with at most 4% more tool calls across the evaluated cells. Inputs that a tool reads from its own state, such as generated tests, are also included in the key.

Active serving then restricts reuse to the resulting allowlist, while state epochs separately prevent reuse across mutations. TwinHash maintains per-resource state epochs, one for each resource a tool declares, such as a database record, a table, a file, or a web page, not one per session. The key includes the epochs of the resources a call reads. An epoch advances when any session writes the resource or its source signals a change, for example through a change log or a file notification. Any change thus produces a new key as long as the signal precedes dependent reads. A predicate read depends on its whole table, and a tool that declares no resource depends on a global epoch that every write advances. Unmodified resources remain at epoch zero and are shared across tasks that start from the same database or repository snapshot, whereas a modified resource is private to the modifying task. A source without a change signal, such as web search, is admitted to SafeTools without profiling, as its reuse is confined to one task and judged by task accuracy in §VI-E. The task scope identifies the snapshot for shared resources and the task in all other cases.

❷ Decode response reuse safety. Decode response reuse should not rely on output comparison as probabilistic sampling intentionally seeks structural divergence. Instead, TwinHash establishes safety via an allowlist *SafeResponses* with three

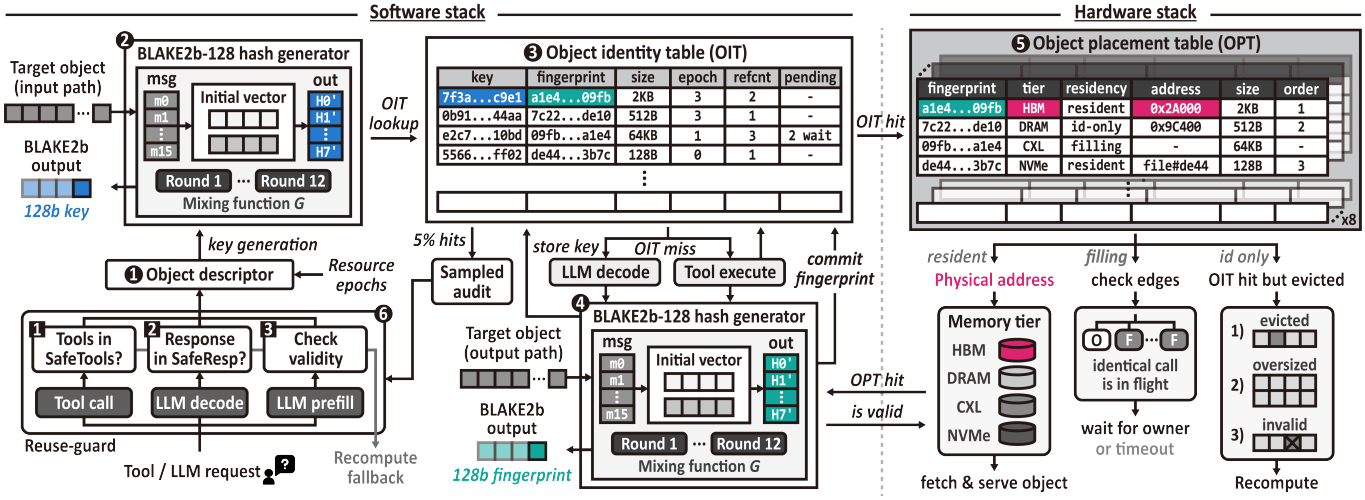

Fig. 8. Architectural overview of TwinHash. Input-side keys identify reusable objects in the OIT, output-side fingerprints validate their content and map physical residency through the OPT, and type-specific reuse guards facilitate safe serving.

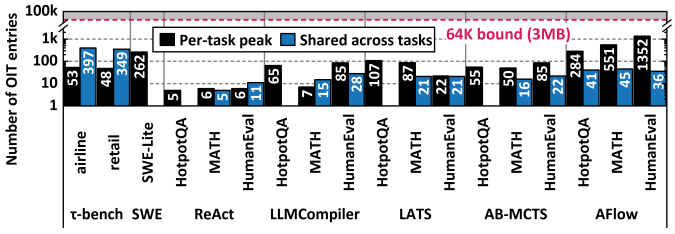

Fig. 9. OIT entries per cell, given as the per-task peak and the pool shared across tasks, against the default 64K-entry (3 MB) bound. Cells without exact per-task counts show the whole-run counts, which are an upper bound.

temporal rules. First, only finalized responses are committed to prevent useless replay of truncated generations. Second, coalescing concurrent and identical requests is prohibited to preserve mandatory diversity of parallel search algorithms [22], [65]. Third, reuse counts are limited for each key to prevent algorithmic insufficiency of the resampling diversity.

❸ KV-prefix reuse safety. KV-segment caching structurally guarantees semantic safety through exact-match keying of model configurations and absolute positional encodings. The primary vulnerability emerges exclusively during cross-tier migrations where logical address bindings occasionally desynchronize from physical bytes. TwinHash verifies every promoted segment against its hash-based content fingerprint.

C. Placement and Lifecycle

The OPT manages overall physical residency of keyed objects across presumed memory subsystems based on *demote-on-eviction* and *promote-on-hit policy*, i.e., tiered-least recently used (LRU). Like a page table, the OPT is the authoritative map that the runtime's movements consult and update, while the tiering substrate beneath it performs the movements. Our demotion-on-eviction policy follows a write-back discipline of conventional systems, such as Linux demoting cold pages toward CXL and PMEM nodes during reclaim, and SGLang's hierarchical cache offers the same write-back mode for KV evicted from GPU HBM. It also departs from offloading methods [33], [45] which copy KV into DRAM proactively at creation and thus consume memory bandwidth for each object by maintaining at least two duplicate KV-segment copies. Conversely, the demotion prevents duplicates as one fingerprint maps to one physical instance. Serving a lower-tier hit forces a fetch since a KV-segment must reach the GPU to be consumed; hence, the OPT hit migrates objects toward fast memory.

A hardware OPT is worthwhile for KV-segments, whereas a software directory suffices for the much smaller tool outputs and LLM responses. Since a framework must locate gigabytes of KV-segments at specific physical tiers, a software-only process can only observe virtual addresses and incur undesired user-space duplicates. The kernel also migrates pages independently, as in NUMA balancing and memory tiering [37], so a kernel driver must follow every migration page by page or pin a dedicated pool. Lookups by the CPU runtime itself need no hardware, since a software table over a pinned pool answers within 16-58 ns. Lookups by devices such as GPUs, however, reach the software table only through the host CPU and take 4.6 μs with a dedicated host core polling at about 3 W, whereas the OPT answers within 2.25 ns while dissipating 3.83-56.4 mW. Moreover, memory pressure from the KV-segments evicts cached tool outputs and LLM responses from HBM, which leads to recompute fallbacks. Here, the table helps sustain TwinHash's hit rate within a session by fetching and promoting the evicted objects from host memory. The fingerprint thus couples each object identity in the OIT with a single physical copy that the OPT locates in any tier.

As Fig. 8 shows, the OPT on the host is designed to serve all tiers, and copy engines of every tier update the OPT without the host CPU. A copy engine updates an entry only after its copy completes, and the DRAM and CXL pools lie outside OS page migration, so a resident entry refers to a complete copy. A referenced entry is not evicted or demoted.

Algorithm 1 TwinHash Object Identity Management

Require: Request r of kind
$\kappa \in \{\text{TOOL}, \text{RESPONSE}, \text{KV}\}$ in trajectory T
Ensure: Output corresponding to r
if $\kappa = \text{TOOL}$ & $\text{tool}(r) \notin \text{SafeTools}$ **then**
  $out \leftarrow \text{ExecuteTool}(r)$
  Advance the epochs of the resources r writes
  return out ▷ Write path; not reused
else if $\kappa = \text{RESPONSE}$ & $\text{decode}(r) \notin \text{SafeResponses}$ **then**
  $out \leftarrow \text{ExecuteLLM}(r)$
  return out ▷ Not admitted; not reused
$key \leftarrow \mathcal{H}(\text{Regularize}(\text{descriptor}_\kappa(r)))$ ▷ Input-side key
if OIT contains a committed entry for key **then**
  $fp \leftarrow \text{OIT}[key].fp$
  if $\text{OPT}[fp].state$ is resident & $\text{check}_\kappa(fp)$ **then**
    return $\text{Fetch}(\text{OPT}[fp])$ ▷ Promote to the top tier
  else
    continue ▷ Identity known, but bytes are lost or unsafe
if key is in flight & Produce_κ is deterministic, non-nested **then**
  Wait for the owner's commit, with a bounded timeout
  if the owner commits successfully **then**
    return the owner's output ▷ One execution per burst
Register key as in flight
$out \leftarrow \text{Produce}_\kappa(r)$ ▷ Execute, decode, or prefill
$fp \leftarrow \mathcal{H}(\text{Regularize}(out))$ ▷ Output fingerprint using the H
$\text{OIT}[key] \leftarrow fp$
$\text{OPT.insert}(fp, out)$
Notify followers
return out

An OPT entry is in one of three residency states to prevent data contamination. A *resident* object resides in a tier and serves read requests via a direct memory fetch. A *filling* object is new, with its bytes still arriving, and blocks concurrent readers until completion. An *identity-only* object retains its OIT presence but has lost its bytes to a bottom-tier eviction or invalidation, so reads fall back to full re-execution of the tool or LLM. The OPT maintains reference counts so that a writer detaches a private copy before mutating a shared object.

D. Serving Integration

We aim to design the unified management scheme for all eligible requests, including tools and LLMs, as in Algorithm 1. The path contains one admission rule and three object-specific decision points, while all other operations are type-agnostic. The admission rule originates *reuse-guard*, which manages reuse safety of each object as detailed in §IV-B.

The first object-specific decision is key construction in §IV-A. As each key includes the state epochs of the resources its call reads, a read issued after a write cannot retrieve an object created before that write. The second decision validates an entry at serving time. For tool reuse hits, retrieved bytes must match the fingerprint recorded at insertion. For response hits, the entry must be finalized and remain within its serving session. Each response is served at most predefined K times, and an exceeding request performs a new decode and replaces the cached response to obtain sample diversity. For KV hits, the segment must match the model configuration and absolute position, and promoted segments are verified with a fingerprint upon arrival. The third decision determines whether simultaneous requests allow coalescing. For allowlisted tools, the primary request executes as the owner and concurrent requests wait for its committed result. Nested tools, which call other cached tools as in LATS on MATH, bypass the wait, since waiting requests could otherwise occupy every worker that the owner's calls need. In contrast, as search-based frameworks fan out identical prompts to obtain multiple samples, coalescing is prohibited to achieve candidate diversity.

Note that TwinHash complements existing prefix caching methods rather than replaces them. Under the validity conditions of §IV-B, the reuse guards make serving a cached object safe, and TwinHash does not mutate a shared prefix.

E. Implementation Costs and Overheads

The overhead of TwinHash can be classified as probe latency (hashing and lookup), metadata storage (OIT), and physical object storage (OPT). In our custom C++/CUDA software kernel of TwinHash, a directory probe costs 24-32 μs per eligible call. As tool executions consume milliseconds to seconds and LLM calls take seconds to minutes, the kernel call overhead is orders of magnitude below the reuse target.

OIT memory footprints remain negligible because the directory tracks key-fingerprint hashes rather than raw content, where each entry requires only tens of bytes for a 16-byte key digest, a 16-byte content fingerprint, and access counters. For the OPT, the prototype packs the 16-byte fingerprint, tier tag, physical location, size, reference count, and version fields into a 261-bit payload, roughly 33 bytes. A 2K-entry OPT managing up to 80 GB of KV-segments therefore requires only 67.5 KB of SRAM. The OPT prototype is 8-way set-associative to mitigate the implementation costs, and a full set demotes its zero-reference tree-PLRU victim. Implemented in RTL and synthesized using 28-nm CMOS operating at a 444 MHz clock, the prototype occupies 0.34 mm^2 while dissipating 3.83-56.4 mW at 0.8 V. As lookup and update operate within a cycle, their latency is at most 2.25 ns. In RTL simulation over 373,510 operations, including agent traces, the synthesized table matches a reference model on every response at one operation per cycle. The functional model of §V also does not report a hit that the hardware table misses.

V. Methodology

Experimental setup. Every experiment runs on a local NVIDIA H200 GPU. Each primary agent policy uses Qwen3-14B on SGLang with radix prefix caching, a static memory fraction of 0.75, and a page size of 1. Thinking is enabled (8K-token context) for HotpotQA, MATH, and HumanEval and disabled (40K tokens) for τ-bench and mini-SWE. Qwen2.5-0.5B, Qwen3 dense models up to 32B, Qwen3.6-27B, and the Qwen3-30B-A3B and Qwen3.6-35B-A3B Mixture-of-Experts models serve as KV-fetch anchors on a separate vLLM with LMCache tiering, where we disable the GPU-internal prefix cache to isolate the lower-tier fetch latency of Fig. 11.

We evaluate five representative agent frameworks with two application agents: τ-bench [59], mini-SWE [23], ReAct [61], LLMCompiler [25], LATS [65], AB-MCTS [22],

and AFlow [63]. Within a unified harness, each policy runs on three benchmarks: HotpotQA [57], MATH [17], and HumanEval [8]. Tools are web or document search on HotpotQA, calculators and WolframAlpha on MATH, code or test generation and execution on HumanEval, database reads and writes in τ-bench, and a shell in mini-SWE. AFlow calls only its scorers. For each framework-benchmark cell, we report the mean over 10 or 15 tasks, with the standard deviation over runs marked. Fig. 4 reports the calls per task. An LLM call averages 0.2K-2.7K prompt tokens on the three benchmarks, 4.3K-7.0K in τ-bench, and 17K in mini-SWE, with 54 to 4.9K output tokens where recorded. Qwen3-14B reaches 0.60-1.00 baseline quality in 11 of the 15 framework cells (exact match or F1 on HotpotQA, accuracy on MATH, and pass rate on HumanEval), enough to expose the workflow structure governing reuse.

We further evaluate τ-bench [59] to measure tool-side reuse over vendor model transcripts. Replaying empirical GPT-4o and Claude-Sonnet transcripts through a function-calling agent under the identity, epoch, and fingerprint rules, we generate 2,809 airline and 8,042 retail tool reads without new model calls. Since these transcripts lack per-call latency data, we use a local Qwen3-14B run for the latency breakdown in Fig. 3. To probe reuse under mutable repository state, we add a terminal-driven coding agent mini-SWE [23] across 15 SWE-bench Lite instances in isolated containers. TwinHash reuses only read-only shell commands, such as cat and grep, and every other command advances the epoch. We further vary three frameworks to isolate how search structure shifts reuse: LATS across per-node samples of 2, 8, and 10, AB-MCTS's adaptive branching on the same tasks, and AFlow's workflow optimizer over 15 validation problems, all on the shared serving stack.

To test whether reuse persists as the hot-tier capacity shrinks, we replay verified event streams against the functional model of our OPT prototype in §IV-C under capacity-scaled memory tiers, separately from the end-to-end runs of Fig. 12, with the identity directory unbounded. Tier service times follow product-class device timings: DDR5 DRAM 0.2μs, 25GB/s; CXL 0.6μs, 12GB/s; NVMe 80μs, 3GB/s.

Across all experiments, we measure end-to-end savings using paired runs that differ only in reuse routing. The *vanilla* mode is each cell's default setup, the baseline. The *observe* mode performs all directory lookups to calibrate SafeTools yet bypasses reuse, whereas the *memoize* mode serves verified cache hits. The vanilla-memoize difference therefore yields TwinHash's overall savings, while the observe-memoize difference isolates the effect of reuse. SGLang enables its radix KV cache identically in all modes, so standard prefix reuse cancels out in the paired ratio, and the reported savings compound on a state-of-the-art prefix-cached baseline. For decode-layer reuse, measured the same way, a reverse proxy manages the response cache by keying on the prompt and sampling controls, excluding the random seed and refusing to coalesce in-flight model calls. On the tool side, we instrument the memoizer at AB-MCTS's search tools and AFlow's deterministic scorers.

Measurement and metrics. We count reuse at two levels for each object type. A *duplicate* in Fig. 5 is a tool output or LLM request that repeats an earlier one, and a *verified* reuse is a tool call whose request key and output fingerprint both repeat. Tool hits match the descriptor key, then the output fingerprint. LLM requests match the model, messages, and sampling parameters except the seed, and KV-prefixes match token IDs and absolute positions. *Verified reuse* in Fig. 6 denotes the fraction of reads whose content check confirms byte-identical output with a previously materialized object. It therefore captures the repeated reads that are demonstrably safe to reuse and serves as an upper bound on the reuse achievable by a live runtime. *Response reuse*, by contrast, measures a substitution rate because sampled LLM responses cannot be validated through byte-level content checks before execution, so we audit it by comparing answer quality between runs with and without substitution. The tool and LLM time shares in Fig. 3 are computed by merging overlapping execution intervals. For tree-search policies executing tool and LLM concurrently, we report the more conservative time share.

Figs. 3-6 use vanilla or observe runs, except the LLM-prompt duplicates of AB-MCTS and AFlow in Fig. 5, counted in the request logs of memoize runs. With TwinHash off, whole prompts that repeat still form 81-91% of the prefix-cached tokens of AB-MCTS. Fig. 10(a) takes verified reuse from replayed calls (τ-bench, mini-SWE, and LATS on HotpotQA), observe runs (AB-MCTS and AFlow), and the cache hits that memoize runs serve after the reuse guards in the other cells. The τ-bench cells replay GPT-4o and Claude-Sonnet transcripts. Fig. 10(b) replays recorded LLM requests, and Fig. 14(b)-(d) replays the response logs of memoize runs. With TwinHash off, the radix cache serves 39-99% of the prompt tokens, so the baseline already includes KV-prefix reuse.

To evaluate energy at different granularities, we measure per-event energy using idle-corrected RAPL measurements averaged over ten repetitions for each domain. The reported *measured task energy*, the end-to-end energy consumed while serving one task, combines tool-layer energy with GPU board energy, computed by integrating board power above idle over the corresponding interval of each paired run. We also characterize energy and latency at the per-hit KV-reuse granularity. KV-fetch speedup is the ratio of prefill latency to the memory-fetch latency to retrieve the corresponding KV state for a single reuse event. Unless otherwise specified, all experiments use one request stream and report per-task latency and energy.

VI. Evaluation

We first analyze reuse per object type, namely external tool outputs, LLM responses, and prefill KV-prefixes, in §VI-A, then quantify end-to-end latency and energy savings in §VI-B and service quality in §VI-C. Lastly, we evaluate the proposed OPT under severe memory pressure in §VI-D and reuse correctness under guards and state drift in §VI-E.

A. Reuse Coverage Across Object Types

Tool output reuse. Fig. 10(a) reports verified reuse, the fraction of tool reads that match a previously materialized

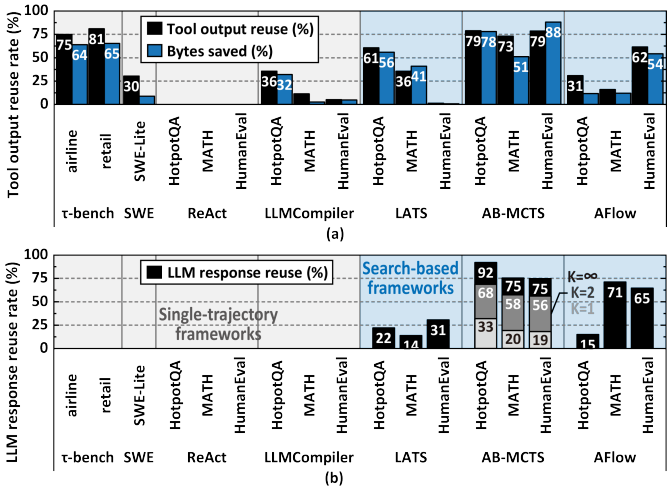

Fig. 10. Reuse coverage across workloads. (a) Verified tool output reuse rate and fraction of the output bytes covered, and (b) LLM response reuse rate.

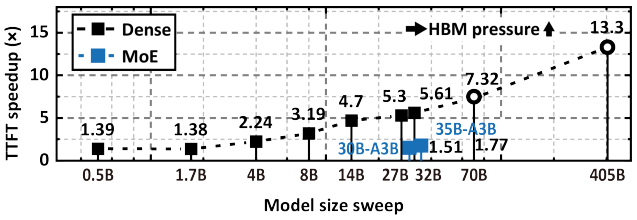

Fig. 11. TTFT speedup from retrieving an evicted KV-prefix rather than recomputing its prefill state, measured for Qwen-family dense and MoE models up to 35B and modeled for 70B and 405B dense models.

object and thus qualify for safe reuse. The database-backed τ-retail and τ-airline workloads repeatedly issue read-only tool calls with identical arguments, for which the tools produce deterministic outputs. Consequently, verified reuse reaches 81.0% and 74.9%, respectively, accounting for 63.9-65.4% of their total tool-output bytes. Cross-task hits alone constitute 34.7% of the tool-output bytes of both domains and therefore fall outside the scope of prefix- or session-scoped caches.

Search-based frameworks also exhibit substantial tool reuse because tree traversal revisits prior queries and repeatedly invokes deterministic scorers. AB-MCTS reaches up to 78.9% verified reuse, covering up to 88.1% of its tool-output bytes, while LATS and AFlow reach 60.6% and 61.5%, respectively. Single-trajectory agents further show that search structure alone does not determine tool reuse. The τ-bench function-calling agent achieves high verified reuse because its database-backed workflow repeatedly issues reads with identical arguments. In contrast, ReAct records 0% verified reuse in our traces because its linear execution path rarely revisits the same tool inputs. Tool reuse therefore depends on both argument recurrence and output determinism, as recurring arguments create reuse opportunities within and across sessions, and deterministic outputs make the opportunities safe to exploit.

For the database-backed τ-bench workloads, verified reuse translates directly into reductions in tool-layer energy and execution time. We measure 869 μJ for a native database read but only 95 μJ for serving its verified result, making native execution 9.1× more energy-intensive. TwinHash reduces tool invocation counts by up to 81.0% on τ-bench and up to 78.9% in AB-MCTS HotpotQA. Because these savings arise from bypassing external tool execution rather than GPU inference, they remain independent of the underlying LLM.

Decode-layer response reuse. Tool-output reuse avoids external tool execution but leaves LLM decoding, a major part of GPU energy, unchanged. *Response reuse* targets this remaining cost by serving a completed response when a subsequent model call repeats both the prompt and the sampling configuration. Fig. 10(b) shows that the resulting hit rate is strongly governed by how often an agent's execution structure revisits identical model inputs. With K of §IV-D unbounded, leaving only the default limit of 25 hits per key per minute, the cache serves 74.7-92% of AB-MCTS's model calls, compared with 15.0-71.2% for AFlow and 13.9-30.7% for LATS. These differences reflect how frequently each search strategy revisits completed states: AB-MCTS's adaptive branching produces frequent exact revisits, AFlow's workflow-level optimization yields fewer, and LATS's fixed-width tree expansion exposes the fewest among the evaluated search-based frameworks.

Single-trajectory frameworks exhibit the opposite behavior. None of the cacheable calls from ReAct, LLMCompiler, and the τ-bench agent repeat within an execution, and none of the coding agent's model calls produce a response hit. Their monotonically evolving histories and one-shot execution plans rarely re-issue an identical prompt under the same sampling configuration. Response reuse is therefore concentrated in search-based executions that revisit evaluated model states, rather than simply in technically cacheable model calls.

Prefill KV-prefix reuse. Beyond tool outputs and decode responses, TwinHash is designed to apply the same identity and placement mechanism to prefill KV-prefixes, allowing a cache hit to bypass redundant prefill computation. As KV retrieval or prefill recomputation must complete before decoding, time-to-first-token (TTFT) quantifies the latency benefit of KV-prefix reuse. Fig. 11 reports the TTFT speedup of fetching a DRAM-resident KV-prefix instead of recomputing its prefill state.

For dense models, evaluated using the Qwen3 and Qwen3.6 families, KV retrieval becomes increasingly advantageous as model size grows, reaching a 5.6× TTFT speedup for the 32B model. The trend arises because, at a fixed prefix length, prefill cost grows with the dense parameter count, whereas the number of transformer layers, KV heads, and cached tokens primarily determines KV-transfer cost. A scaling model calibrated to our measured results and GQA geometry projects TTFT speedups of 7.3× at 70B and 13.3× at 405B.

The benefit scales less for Mixture-of-Experts (MoE) models from the same families. While prefill activates only a subset of the parameters, KV retrieval must still transfer cached state across all transformer layers. Consequently, the evaluated 30B and 35B MoE models achieve a smaller 1.5-1.8× TTFT speedup than the 5.6× of the dense 32B model.

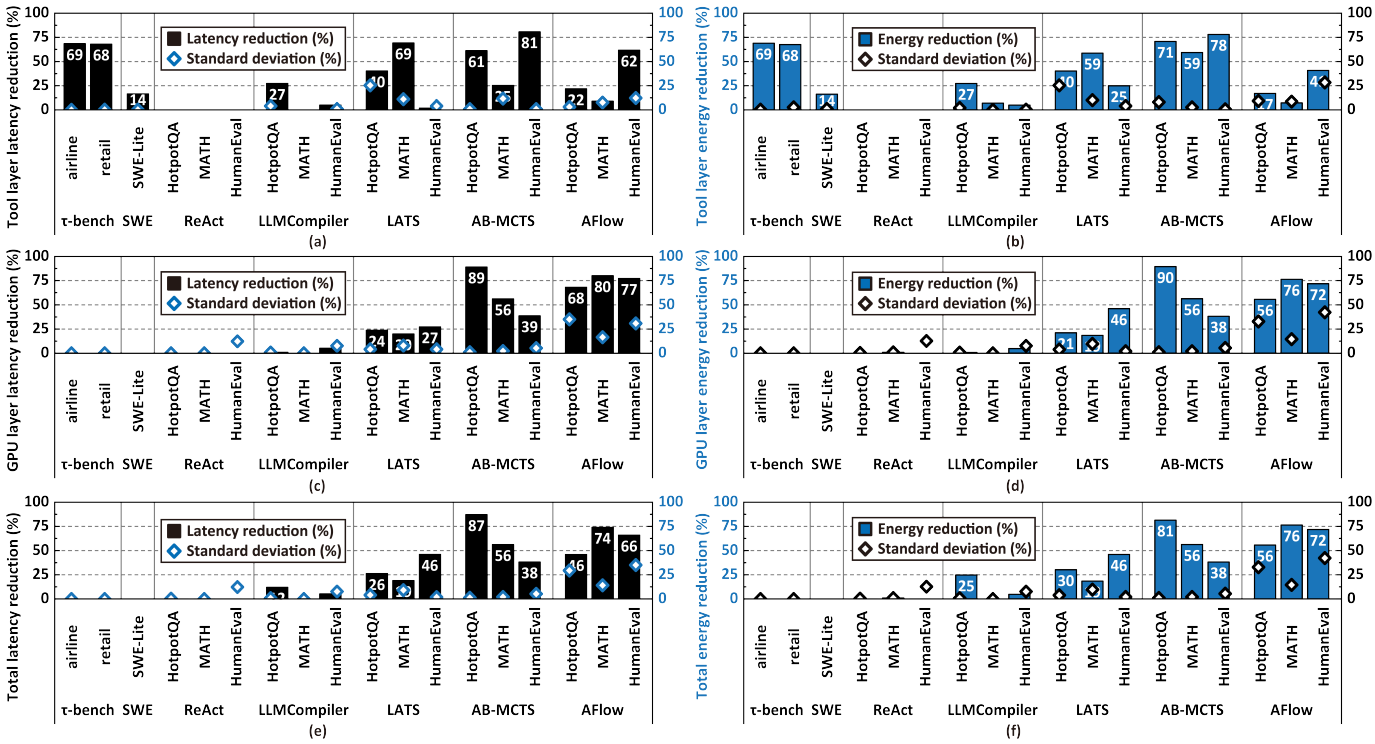

Fig. 12. Average latency and energy reductions from TwinHash reuse across agent frameworks and benchmarks, measured on Qwen3-14B in our local serving stack, where the per-key serving cap is set to $K = 2$ for AB-MCTS MATH to retain task quality. (a) Tool-layer latency reduction, (b) tool-layer energy reduction, (c) GPU-layer latency reduction, (d) GPU-layer energy reduction, (e) end-to-end latency reduction, and (f) end-to-end energy reduction.

B. Latency and Energy Savings

Fig. 12 quantifies the latency and energy savings from reuse at the tool, GPU, and end-to-end layers, measured as the paired observe-memoize difference. As both modes use the same radix prefix cache, all reported gains are measured on top of a state-of-the-art prefix-cached baseline [64]. Diamonds mark the bootstrap standard deviation over five runs of each cell.

At the tool layer, Fig. 12(a) and (b) show that latency and energy reductions closely follow the verified tool-reuse rates of §VI-A. The largest savings arise in workloads with recurring deterministic tool calls, including the database-backed τ-bench and search-based frameworks with repeated scoring, whereas frameworks with little tool-input recurrence obtain correspondingly limited benefit. At the GPU layer, Fig. 12(c) and (d) show a sharper separation: savings are concentrated in search-based frameworks because response hits bypass entire decodes. Single-trajectory agents show essentially negligible GPU-layer reduction because their prompts do not repeat. Fig. 12(c) sums the durations of all LLM calls, which overlap under concurrent sampling, rather than wall-clock time.

These layer-specific savings combine into the end-to-end results in Fig. 12(e) and (f). AB-MCTS achieves the largest reductions, saving up to 87% of wall-clock time and 81% of measured task energy. AFlow reduces wall-clock time by 46-74% and energy by 56-76%, while LATS reduces both by 19-46%. AFlow exhibits the largest variation because its optimizer rewrites the workflow itself between runs, whereas LATS and AB-MCTS resample candidates within a fixed workflow. Among single-trajectory agents, LLMCompiler on HotpotQA achieves notable end-to-end savings, reducing wall-clock time by 12.1% and energy by 24.7%, since web-access API calls cost a significant 1,836 J of serving energy in total. ReAct shows task energy savings from -3.2% to 1.0%, with bootstrap 5-95% intervals that all include zero. Only 1 of 45 tool outputs in its three cells of Fig. 5 repeats an earlier one, and TwinHash adds under 0.02% to task time. The τ-bench and mini-SWE agents benefit only at the tool layer, where reuse reduces tool latency and energy by 68-69% and 14%, respectively. In mini-SWE, most of the saving arises from two of the 15 instances that repeat the same searches.

C. Service Quality and Response Diversity

In Fig. 13, service quality remains stable under reuse for most agent-workload pairs. Greedy-decoding agents, including ReAct and LLMCompiler, exhibit no quality degradation, nor does τ-bench in its stochastic reward metric. Quality loss arises primarily in adaptive search with stochastic decoding, where repeated sampling of an identical prompt is intentionally used to explore diverse candidate responses. As discussed in §IV-B, the per-key serving cap can be tuned to preserve the diversity while retaining reuse benefits. For AB-MCTS on MATH, setting K=2 reduces the quality loss from 0.25 to 0.04 while still reducing time and energy by 56.1% and 56.4%, compared with 84.1% and 84.6% without a cap. Although K=1 achieves the same measured accuracy, it reduces time and energy by

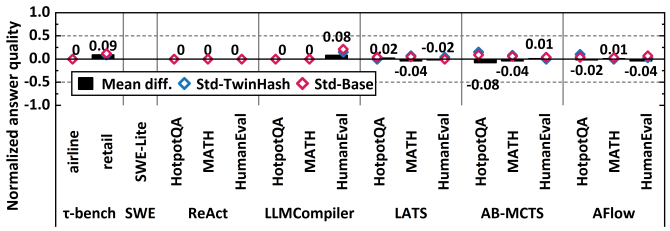

Fig. 13. Change in normalized task quality under TwinHash reuse relative to the paired no-reuse configuration across agent frameworks and benchmarks.

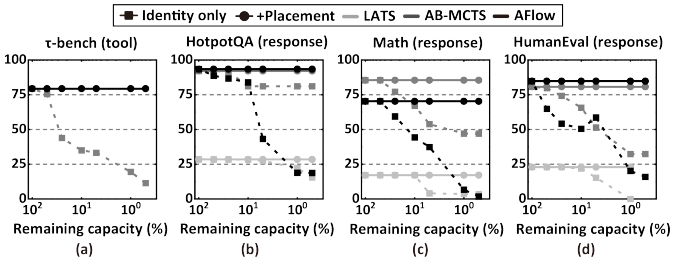

Fig. 14. Object hit rate under increasing hot-tier memory pressure for (a) τ-bench tool objects and (b)-(d) HotpotQA, MATH, and HumanEval response objects. Identity-only serving loses reusable state as HBM capacity shrinks, whereas identity-plus-placement sustains each policy's reuse ceiling.

only 18.2% and 19.0%. The number of forced fresh decodes per run increases from 5 to 39 and then to 63 as K tightens.

D. Placement Under Pressure

KV-segments dominate hot-tier capacity, but evicting much smaller tool outputs and model responses can eliminate disproportionately valuable reuse. To isolate the benefit of placement, Fig. 14 compares identity-only serving, where an OIT hit without resident bytes triggers re-execution, against identity-plus-placement, where the OPT retrieves the object from a lower tier. The functional model of our OPT prototype in §V produces the results with presumed product-class tier timings. As an OPT entry records only a location and a size, the OPT handles tool and response objects like KV-segments.

Hierarchical placement becomes increasingly valuable as the reusable working set exceeds hot-tier capacity. For τ-bench tool objects, Fig. 14(a) shows the identity-only hit rate falling to 33.3% at 5% remaining capacity and 19.5% at 1%, whereas identity-plus-placement sustains the 79.4% verified-reuse ceiling throughout. In Fig. 14(b)-(d), identity-only serving also progressively loses reusable response objects, while placement preserves each framework's response-reuse ceiling. The benefit is largest for search-based policies with high structural reuse, where eviction otherwise destroys a larger fraction of reusable computation. On τ-bench, placement-driven energy savings increase from 3.3% at 2× pressure to 59.2% at 200×.

Live measurements further validate the modeled tier hierarchy. Fetching an evicted KV-prefix from CPU DRAM outpaces prefill recomputation by 1.8-6.0× in TTFT across 0.8K-6.4K-token prompts on an 8B model, supporting DRAM as an effective lower-tier reuse target, whereas disk retrieval achieves 0.86-1.3×, indicating that the bottom tier is more valuable for high-capacity retention than for latency-optimized serving. As severe memory pressure is primarily driven by growing KV state during extended sessions, the value of placement further increases as model and context sizes scale [7], [9], [38].

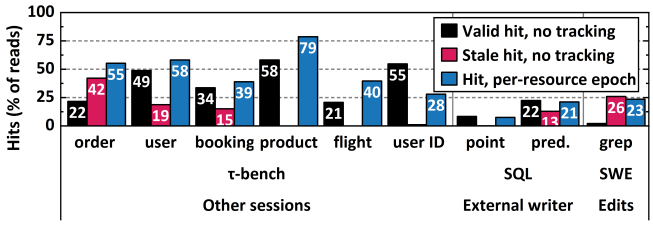

Fig. 15. Hits per tool type of replayed calls while other sessions, an external writer, or out-of-band edits change the state, with stale hits in red.

E. Reuse Guard Ablation and State Drift

Removing one reuse-guard rule at a time shows what each rule prevents. Without output checks, an argument-only cache serves wrong bytes on 251 of 10,330 hits. Without task scopes, shell reads shared across SWE-bench repositories are wrong on all 126 hits, and without state epochs, 22 of 103 reads repeated across a write differ. With all rules, the cache serves no wrong hit. For responses, without the default limit of §VI-A, 5 of 9 LATS runs stalled on a single key, and coalescing sampled calls collapsed each 5-way fan-out into one decode. Our prototype implements the validity rules at low cost, as task-local scope changes only 4 of 1,695 HotpotQA reuse decisions and 5% audits find no mismatch while re-executing up to 4% of tool calls. Including audit energy would lower the savings of Fig. 12(b) by at most 4 points.

Fig. 15 evaluates whether per-resource state epochs prevent stale hits in three replays with state drift. With every change signaled before dependent reads, the epochs serve no stale hit and still hit 7-79% of reads. In τ-bench, 100 airline and 230 retail sessions share one database per domain (21-24% of tool calls write). The SQL of 4,495 DBBench tasks [32] queries 152 databases, each shared by eight interleaved sessions. An external writer changes one row of a random table after each statement with probability 0.5. In mini-SWE, 114 out-of-band edits per run each insert a commented-out copy of a line that is read later. The *no tracking* baseline sees only a session's own writes (τ-bench), writes through the cache (SQL), or the agent's own commands (mini-SWE), and up to 66% of its database hits and 93% of its file hits are stale. Compared with a per-session epoch, per-resource epochs preserve sharing after a session's first write (58% to 79% for product lookups), but any change to a scanned record invalidates a table scan (55% to 28% for user-ID lookups). In Fig. 10(a), τ-bench shares only reads before a session's first write, and per-resource epochs would raise both rates to 83-88% without a stale hit.

Conversely, web results change without a signal. Over 65 days, 28.6% of queries returned different results, whereas 97% of repeated searches within a task returned identical results. LLMCompiler on HotpotQA, which reuses only web search, still retains a task quality of 0.60 with and without reuse.

VII. Discussion and Limitations

LLM response reuse and sampling diversity. On the AB-MCTS MATH benchmark, unbounded reuse of stored samples under stochastic decoding lowers accuracy from 0.90 to 0.65. As §VI-C shows, a tighter cap than K=2 sacrifices most of the efficiency gain with little additional quality benefit, and each fresh decode replaces the stored sample and restores sampling diversity. This degradation is thus specific to stochastic decoding, where repeated sampling contributes to exploration.

Time and energy crossovers for KV fetching. KV-segment reuse exhibits different time and energy crossovers at the NVMe tier. For Qwen3-8B, an NVMe-tier fetch is faster for the 848-token prompt (1.31×), and recomputation is faster beyond roughly 1.6K tokens, with recompute-to-fetch time ratios of 0.95× at 1,645, 0.99× at 3,244, and 0.86× at 6,447 tokens. A cold prefill consumes 72.45 J, whereas a tier hit consumes 1.04 J. Recomputation still draws 300-400 W of net GPU board power against an estimated 10-15 W for an NVMe fetch, so the fetch uses an order of magnitude less energy even where recomputation is faster. The latency crossover thus lies between the modeled CXL tier and NVMe, and the energy crossover beyond NVMe. Increasing model size shifts the latency crossover further in favor of fetching, as prefill cost scales more rapidly with model size than KV transfer.

Benefits beyond search-based execution. Single-trajectory agents repeat no response, yet tool reuse still saves up to 12.1% of their time and 24.7% of their energy, and the saving grows with tool requests that recur across tasks and users. Evicted KV-prefix retrieval applies to every framework, as the single-trajectory agents of §II-A also append to a shared context, and the prefix cache serves 52-99% of their prompt tokens.

Scaling beyond a single node. A cluster can shard the OIT by key hash, with each owner shard storing the in-flight state of its keys. Each node's OPT treats peer memory as a lower tier, as in distributed KV pools [45]. In a trace-driven τ-bench simulation, task-affine routing over eight node-local directories retains 96% of the reuse, and key sharding loses no reuse.

Limitations. Per-resource state epochs assume that change signals precede dependent reads. Sources without a change signal, such as web search, are reused only within a task. We validate the OPT prototype through synthesis and trace-driven simulation, while silicon or FPGA integration, physical CXL memory, and multi-node deployment remain future work.

VIII. Related Work

Agentic frameworks and inference-time scaling. CoT [54] introduces intermediate reasoning steps, while ToT [60] and GoT [4] organize them into tree- and graph-structured representations. ReAct [61] interleaves reasoning with actions and observations, LLMCompiler [25] generates dependency-aware function-call plans, and multi-agent frameworks [19], [30], [44], [55] coordinate role-specialized collaborators. Search-based systems [2], [15], [21], [22], [62], [65] further explore multiple reasoning or execution paths, repeatedly invoking tools and models. TwinHash complements these frameworks without modifying reasoning or search policies, instead reclaiming repeated work exposed by their execution structures.

Agent-aware scheduling. Parrot [31] and Autellix [36] schedule agent programs to reduce queueing delays and context churn. KVFlow [43] prefetches predicted KV state, FlashAgents [12] overlaps decoding with prefilling, and PASTE [51] and TokenCake [5] mitigate tool-induced stalls through speculation and wait-aware KV-cache management. These techniques reduce latency or memory pressure through scheduling, prefetching, speculation, and overlap, whereas TwinHash avoids verified duplicate execution.

Model-side serving reuse. vLLM [27], SGLang [64], Prompt Cache [13], and CAG [7] reuse compatible prefixes and position-aware or preloaded model state, while CacheBlend [58] extends KV reuse to retrieval-augmented inputs [29]. Other systems move model parameters or KV state across memory tiers, hosts, and storage [1], [20], [28], [33], [34], [45], [49]. In contrast, TwinHash generalizes reuse beyond model-native state to tool outputs, model responses, and KV segments through a unified identity abstraction.

Content-addressed storage and build caches. Content-addressed stores identify data by a hash of its bytes [41], [46], [66], and copy-on-write file systems share blocks until a write [47], as the OPT reference counts do. Build caches key outputs by a hash of their inputs [10], [11], [16], [40], as does memoization [39]. Input-keyed caches assume hermetic producers, which agentic tools and sampled decoding are not. TwinHash extends both hashes with versions, state epochs, audits, and serving caps as rules for valid reuse.

IX. Conclusion

As agentic AI shifts from isolated model calls to dynamically constructed workflows, serving efficiency increasingly depends on eliminating repeated work rather than merely accelerating each invocation. TwinHash addresses this opportunity through a unified object-management architecture that spans tool outputs, LLM responses, and KV prefixes using input-side identities, output-side fingerprints, type-specific reuse guards, and hierarchical placement metadata. Across seven agent frameworks and five benchmark suites, we find that reuse is governed primarily by workflow structure rather than benchmark identity, with TwinHash serving up to 81% of tool outputs and 92% of LLM responses. By independently eliminating redundant work at the tool and GPU layers, TwinHash converts these reuse opportunities into end-to-end reductions of up to 87% in wall-clock time and 81% in measured task energy over a prefix-cached baseline. Extending the same identity to object placement further preserves reusable state under severe memory pressure, increasing the object hit rate from 19.5% to 79.4% at 1% resident capacity. The hardware OPT records object placement across all tiers without the host CPU, within 2.25 ns at 3.83-56.4 mW. Therefore, TwinHash unifies object management across heterogeneous objects, establishing a cost-effective and scalable framework for the emerging era of complex agentic workflows.

REFERENCES

[1] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 12 562–12 584.

[2] A. Antoniades, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang, "SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement," in *International Conference on Learning Representations*, 2025.

[3] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 2023, pp. 212–218.

[4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," in *AAAI Conference on Artificial Intelligence*, 2024.

[5] Z. Bian, F. Wu, Z. Li, T. Ma, and Y. Zhuo, "TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications," *arXiv preprint arXiv:2510.18586*, 2025.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020, pp. 1877–1901.

[7] B. J. Chan, C.-T. Chen, J.-H. Cheng, and H.-H. Huang, "Don't do RAG: When cache-augmented generation is all you need for knowledge tasks," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 893–897.

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[9] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[10] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Large Installation System Administration Conference*, 2004, pp. 79–92.

[11] H. Esfahani, J. Fietz, Q. Ke, A. Kolomiets, E. Lan, E. Mavrinac, W. Schulte, N. Sanches, and S. Kandula, "CloudBuild: Microsoft's distributed and caching build service," in *International Conference on Software Engineering Companion*, 2016, pp. 11–20.

[12] T. Fang, Z. Zheng, Z. Ma, Y. Lu, H. Lin, X. Han, and L. Sun, "FlashAgents: Accelerating multi-agent LLM systems via streaming prefill overlap," in *Proceedings of Machine Learning and Systems*, 2026.

[13] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," in *Proceedings of Machine Learning and Systems*, 2024.

[14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, G. Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar,

V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Y. Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The Llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[15] X. Guan, L. L. Zhang, Y. Liu, N. Shang, Y. Sun, Y. Zhu, F. Yang, and M. Yang, "rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking," in *International Conference on Machine Learning*, 2025.

[16] P. K. Gunda, L. Ravindranath, C. A. Thekkath, Y. Yu, and L. Zhuang, "Nectar: Automatic management of data and computation in datacenters," in *USENIX Symposium on Operating Systems Design and Implementation*, 2010, pp. 75–88.

[17] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the math dataset," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[18] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 6609–6625.

[19] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024.

[20] C. Hu, H. Huang, J. Hu, J. Xu, X. Chen, T. Xie, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "MemServe: Context caching for disaggregated LLM serving with elastic memory pool," *arXiv preprint arXiv:2406.17565*, 2024.

[21] Y. Hu, Y. Zhao, C. Zhao, and A. Cohan, "MCTS-RAG: Enhancing retrieval-augmented generation with monte carlo tree search," in *Findings of the Association for Computational Linguistics: EMNLP*, 2025, pp. 12 581–12 597.

[22] Y. Inoue, K. Misaki, Y. Imajuku, S. Kuroki, T. Nakamura, and T. Akiba, "Wider or deeper? scaling LLM inference-time compute with adaptive branching tree search," in *Advances in Neural Information Processing Systems*, 2025.

[23] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations*, 2024.

[24] J. Kim, B. Shin, J. Chung, and M. Rhu, "The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2026, pp. 1–16.

[25] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM compiler for parallel function calling," in *International Conference on Machine Learning*, 2024.

[26] Kimi Team, T. Bai, Y. Bai, Y. Bao, M. C., J. Cai, X. Cai, P. Cao, Y. Cao, Z. Chai, Y. Charles, H. S. Che, G. Chen, G. Chen, G. Chen, H. Chen, J. Chen, J. Chen, J. Chen, K. Chen, P. Chen, R. Chen, W. Chen, X. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, D. Cheng, Y. Cheng, J. Cui, J. Cui, A. Dai, J. Deng, H. Ding, R. Ding, S. Ding, M. Dong, M. Dong, Y. Dong, Y. Dong, A. Du, C. Du, D. Du, J. Du, Y. Du, Y. Fan, J. Feng, Q. Feng, Y. Feng, K. Fu, Q. Fu, F. Gao, H. Gao, J. Gao, T. Gao, W. Gao, S. Geng, J. Gong, L. Gong, S. Gong, X. Gong, Q. Gu, Y. Gu, S. Guan, H. Guo, S. Guo, X. Guo, Z. Guo, B. Hao, W. Hao, X. Hao, D. He, H. He, L. He, Q. He, W. He, X. He, X. He, Y. He, Y. He, C. Hong, T. Hong, H. Hu, J. Hu, R. Hu, W. Hu, Y. Hu, Z. Hu, L. Hua, J. Huang, K. Huang, R. Huang, S. Huang, W. Huang, Y. Huang, Z. Huang, Z. Huang, Y. Hui, C. Jia, Y. Jiang, Z. Jiang, Z. Jiang, W. Jin, X. Jin, Y. Jing, H. Kong, G. Lai, A. Li, C. Li, C. Li, C. Li, F. Li, G. Li, H. Li, J. Li, J. Li, L. Li, L. Li, L. Li, W. Li, W. Li, X. Li, Y. Li, Y. Li, Y. Li, Y. Li, Z. Li, Z. Li, Z. Li, Z. Li, Z. Li, J. Lin, X. Lin, Y. Lin, Z. Lin, Z. Lin, B. Liu, B. Liu, C. Liu, L. Liu, S. Liu, S. Liu, S. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, Z. Liu, E. Lu, H. Lu, L. Lu, T. Lu, Z. Lu, A. Luo, G. Luo, J. Luo, Y. Luo, B. Lyu, W. Lyu, S. Mao, Y. Mei, X. Men, M. Ni, Y. Niu, S. Pan, S. Peng, Z. Qi, R. Qin, Z. Qin, Z. Qin, H. Qiu, J. Qiu, J. Qiu, B. Qu, Y. Qu, Z. Shang, Y. Shao, H. Shen, J. Shi, J. Shi, L. Shi, S. Shi, W. Siu, P. Song, X. Song, J. Su, Y. Su, Z. Su, L. Sui, J. Sun, J. Sun, S. Sun, S. Sun, T. Sun, Y. Sun, Y. Tai, C. Tang, H. Tang, S. Tang, Z. Tang, C. Tian, R. Tian, Y. Tian, W. Tu, C. Wang, C. Wang, C. Wang, D. Wang, F. Wang, H. Wang, H. Wang, H. Wang, H. Wang, H. Wang, H. Wang, J. Wang, J. Wang, J. Wang, L. Wang, S. Wang, S. Wang, S. Wang, S. Wang, T. Wang, W. Wang, X. Wang, X. Wang, X. Wang, X. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Z. Wang, Z. Wang, Z. Wang, Z. Wang, Z. Wang, C. Wei, M. Wei, S. Wei, Z. Wen, F. Wu, H. Wu, R. Wu, W. Wu, X. Wu, Y. Wu, Y. Wu, Y. Wu, Z. Wu, X. Xian, C. Xiang, Y. Xiang, B. Xiao, C. Xiao, X. Xiao, J. Xie, X. Xie, Y. Xie, Z. Xie, B. Xing, Y. Xiong, B. Xu, B. Xu, J. Xu, J. Xu, J. Xu, J. Xu, L. H. Xu, Q. Xu, S. Xu, S. Xu, T. Xu, T. Xu, W. Xu, X. Xu, Y. Xu, Y. Xu, Y. Xu, Z. Xu, H. Xue, J. Yan, Y. Yan, F. Yang, G. Yang, H. Yang, J. Yang, R. Yang, W. Yang, X. Yang, X. Yang, Y. Yang, Y. Yang, Y. Yang, Y. Yang, Z. Yang, Z. Yang, Z. Yang, Z. Yang, H. Yao, D. Ye, H. Ye, W. Ye, Z. Ye, B. Yin, H. Yin, X. Yin, C. Yu, H. Yu, L. Yu, S. Yu, S. Yu, T. Yu, E. Yuan, M. Yuan, T. Yue, W. Yue, Y. Yue, D. Zha, H. Zhan, B. H. Zhang, D. Zhang, F. Zhang, H. Zhang, H. Zhang, H. Zhang, J. Zhang, J. Zhang, J. Zhang, K. Zhang, M. Zhang, P. Zhang, Q. Zhang, R. Zhang, R. Zhang, S. Zhang, S. Zhang, X. Zhang, X. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Z. Zhang, Z. Zhang, B. Zhao, C. Zhao, F. Zhao, J. Zhao, J. Zhao, S. Zhao, W. Zhao, X. Zhao, X. Zhao, Y. Zhao, Z. Zhao, H. Zheng, H. Zheng, R. Zheng, S. Zheng, T. Zheng, H. Zhong, L. Zhong, L. Zhong, M. Zhou, Q. Zhou, R. Zhou, R. Zhou, X. Zhou, Y. Zhou, Z. Zhou, J. Zhu, L. Zhu, X. Zhu, Y. Zhu, Y. Zhu, Z. Zhu, C. Zhuang, W. Zhuang, and X. Zu, "Kimi K3: Open frontier intelligence," 2026. [Online]. Available: https://arxiv.org/abs/2607.24653

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *ACM Symposium on Operating Systems Principles*, 2023.

[28] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *USENIX Symposium on Operating Systems Design and Implementation*, 2024, pp. 155–172.

[29] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474.

[30] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems*, 2023.

[31] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[32] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024.

[33] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," *arXiv preprint arXiv:2510.09665*, 2025.

[34] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *ACM SIGCOMM Conference*, 2024, pp. 38–56.

[35] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "A dynamic LLM-powered agent network for task-oriented agent collaboration," in *Conference on Language Modeling*, 2024.

[36] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, "Agentix: An efficient serving engine for LLM agents as general programs," in *USENIX Symposium on Networked Systems Design and Implementation*, 2026, previously circulated as "Autellix" (arXiv:2502.13965).

[37] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, "TPP: Transparent page placement for CXL-enabled tiered-memory," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023, pp. 742–755.

[38] Meta AI, "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," https://ai.meta.com/blog/llama-4-multimodal-intelligence/, 2025, llama 4 Maverick: 400B total / 17B active (MoE).

[39] D. Michie, "'Memo' functions and machine learning," *Nature*, vol. 218, no. 5136, pp. 19–22, 1968.

[40] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, pp. 79:1–79:29, 2018.

[41] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *ACM Symposium on Operating Systems Principles*, 2001, pp. 174–187.

[42] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[43] Z. Pan, A. Patel, Y. Shen, Z. Hu, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, "KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows," in *Advances in Neural Information Processing Systems*, 2025.

[44] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 174–15 186.

[45] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — A KVCache-centric architecture for serving LLM chatbot," in *USENIX Conference on File and Storage Technologies*, 2025, pp. 155–170.

[46] S. Quinlan and S. Dorward, "Venti: A new approach to archival storage," in *USENIX Conference on File and Storage Technologies*, 2002, pp. 89–102.

[47] O. Rodeh, J. Bacik, and C. Mason, "BTRFS: The Linux B-tree filesystem," *ACM Transactions on Storage*, vol. 9, no. 3, pp. 9:1–9:32, 2013.

[48] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, 2023.

[49] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Re, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *International Conference on Machine Learning*, 2023, pp. 31 094–31 116.

[50] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2023.

[51] Y. Sui, H. Zhao, R. Ma, Z. He, H. Wang, J. Li, and Y. Yang, "Parallelizing tool execution and LLM generation for low-latency agent serving," *arXiv preprint arXiv:2603.18897*, 2026.

[52] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[53] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[54] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022.

[55] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *First Conference on Language Modeling*, 2024.

[56] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, 2024.

[57] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Conference on Empirical Methods in Natural Language Processing*, 2018.

[58] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *European Conference on Computer Systems*, 2025.

[59] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations*, 2025.

[60] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, 2023.

[61] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[62] X. Yu, B. Peng, V. Vajipey, H. Cheng, M. Galley, J. Gao, and Z. Yu, "ExACT: Teaching AI agents to explore with reflective-MCTS and exploratory learning," in *International Conference on Learning Representations*, 2025.

[63] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *International Conference on Learning Representations*, 2025.

[64] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, 2024.

[65] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *International Conference on Machine Learning*, 2024.

[66] B. Zhu, K. Li, and H. Patterson, "Avoiding the disk bottleneck in the Data Domain deduplication file system," in *USENIX Conference on File and Storage Technologies*, 2008, pp. 269–282.